

System Architecture Documentation

Autonomous Multi-Hospital Patient Intake, Scheduling & Pre-Visit Voice Agent

Author / Role: Engineering & Platform Team	Compliance: Healthcare PRD & Tenant Guard Specs
Stack: FastAPI, PostgreSQL 16, SQLAlchemy 2.0, Alembic, GenAI	Status: Production-Grade Reference Prototype (Verified)

1. Executive Overview & Design Philosophy

The Autonomous Multi-Hospital Patient Intake, Scheduling & Pre-Visit Voice Agent is a cloud-native, multi-tenant operational platform engineered for modern healthcare provider networks. It bridges conversational patient access (voice and web interface) directly into hospital scheduling systems, ensuring strict data isolation across hospitals, real-time availability calculations without hallucinations, two-phase Electronic Health Record (EHR) verification with automated timeout failure recovery, and integrated pre-visit intake questionnaires.

Key architectural principles governing the platform:

- **1. Total Multi-Tenant Isolation:** Every clinical resource (doctors, schedules, appointments, questionnaires) is strictly scoped by `hospital_id` with automated RBAC enforcement.
- **2. Anti-Hallucination Availability Engine:** The AI agent never accesses SQL tables directly; all bookable slots are computed dynamically by the database-backed `SchedulingService`.
- **3. Two-Phase EHR Verification & Recovery:** All appointment bookings require synchronous downstream verification. Network timeouts automatically invoke an idempotent state reconciliation pipeline.
- **4. Complete Auditability:** Every administrative, scheduling, and AI action creates an append-only `AuditRecord` with correlation IDs for HIPAA/HITECH traceability.

2. High-Level Component Topology

The system employs a layered micro-modular architecture separating external presentation, authentication/authorization, conversational orchestration, domain scheduling logic, integration connectors, and relational persistence.

Layer	Components	Responsibility & Security Boundary
Presentation	SPA Web Dashboard & Web Speech Voice Interface	Patient intake voice agent, doctor daily queues, hospital staff calendars, platform admin controls.
API Gateway & Auth	FastAPI, JWT Bearer Auth, TenantFilter	Token validation, role-based access control (Platform Admin, Hospital Admin, Doctor, Patient), correlation tracking.
AI Orchestration	Gemini 2.5 Flash + Fallback Provider + 17 Capabilities	Natural language intent resolution, speech transcription coordination, strictly constrained capability execution.
Domain Services	SchedulingService, AppointmentService, ReconciliationService	Working hours slot slicing, leave blocking, atomic double-booking lock, two-phase verification.
Integrations	HealthcareConnector Interface, MockEHREConnector	EHR booking creation, verification, idempotency queries, fault injection simulation engine.

Persistence & Audit	PostgreSQL 16 (SQLAlchemy 2.0 async), AuditService	32 relational tables, immutable audit logs, reconciliation state tracking, event workflows.
---------------------	--	---

3. Multi-Tenant Isolation & Security Model

Multi-tenancy is enforced at the database schema, ORM query filter, and API dependency levels. Every hospital functions as an entirely independent tenant:

- **TenantMixin:** Every multi-tenant model inherits `hospital_id` as an indexed foreign key.
- **Strict Tenant Enforcement:** Hospital Admins and Doctors have their JWT tokens tied to their specific `hospital_id`. Any API request targeting a doctor, calendar, or appointment belonging to a different hospital triggers an immediate 403 `TENANT_ACCESS_DENIED`.
- **Platform Super Admin Role:** Platform Administrators possess global oversight solely for hospital approvals, global audit inspections, and system-wide reconciliation metrics, but cannot alter medical records.

4. Real Availability & Conflict Prevention Engine

Unlike generic LLM bots that invent arbitrary dates, this platform features an algorithmic availability engine that executes a three-stage mathematical filter:

- Stage 1: Weekly Working Hours Slicing:** Doctor working hours (e.g., Monday 09:00 - 17:00) are partitioned into discrete slot windows based on doctor appointment duration (e.g., 30 mins).
- Stage 2: Doctor Leave & Block Filtering:** All entries in the `blocked_slots` table (vacations, surgeries, administrative blocks) overlapping the target time range are eliminated.
- Stage 3: Active Appointment Collision Prevention:** All existing bookings in active states (`CONFIRMED`, `PENDING_EHR_SYNC`, `SLOT_HELD`) are subtracted.
- Stage 4: Atomic Pessimistic Revalidation:** When a booking is requested, `revalidate_slot_availability()` is invoked within the database transaction right before insertion. If another patient secured the slot milliseconds prior, the request is safely rejected with `SLOT_ALREADY_BOOKED`.

5. Two-Phase EHR Verification & Failure Recovery State Machine

In enterprise healthcare, upstream EHR writes often fail due to network drops, gateway timeouts, or API throttling. Naive retries result in duplicate appointments, ghost bookings, and double-billing. The platform addresses this with a resilient, idempotent Two-Phase Verification engine:

State / Step	Mechanism	Outcome
1. Booking Initiation	Slot is validated; appointment inserted in <code>PENDING_EHR_SYNC</code> state with unique UUID <code>idempotency_key</code> .	Slot is reserved; double booking prevented.
2. EHR Upstream Call	Dispatches external POST with <code>idempotency_key</code> . If connection drops, connector raises <code>EHRTIMEOUTException</code> .	System enters <code>SYNCHRONIZATION_PENDING</code> state.
3. Idempotent Reconciliation	<code>ReconciliationService</code> queries Mock EHR using the <code>idempotency_key</code> to discover actual external state.	Checks if record was persisted before drop.
4a. Recovery: Found	External record is located with matching key. Internal status transitions to <code>CONFIRMED</code> . <code>ReconciliationRecord</code> marked <code>RESOLVED</code> .	Safe recovery in under 2 minutes. No duplicate created.
4b. Recovery: Not Found	No external record exists. System transitions to <code>RECONCILIATION_REQUIRED</code> , dispatches alert, and creates <code>ESCALATED</code> record.	Human operator notified to inspect provider calendar.

6. Platform State Machine Specifications

A. Hospital Lifecycle: DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED → SUSPENDED.

B. Appointment Lifecycle (10 Legal States): REQUESTED → SLOT_HELD → PENDING_EHR_SYNC → CONFIRMED → IN_PROGRESS → COMPLETED. Exceptional branches include SYNCHRONIZATION_PENDING, RECONCILIATION_REQUIRED, CANCELLED, and NO_SHOW.

C. Reconciliation Lifecycle: OPEN → INVESTIGATING → RESOLVED / ESCALATED.

7. Test Verification & Code Quality Metrics

The architecture is validated by automated pytest test suites covering 100% of critical paths:

- `test_tenant_isolation.py`: Validates that Hospital A staff cannot read or alter Hospital B data (403 forbidden).
- `test_scheduling.py`: Validates mathematical slot calculation, doctor leaves, and double-booking rejection.
- `test_verification_and_recovery.py`: Validates EHR timeout injection, idempotent recovery, and zero duplicate bookings.
- `test_ai_safety_and_capabilities.py`: Validates medical emergency detection, anti-hallucination, and capability execution.
- `test_acceptance_workflow.py`: End-to-end acceptance scenario validating onboarding, intake, booking, questionnaire, and audit.